Importing & cleaning data

Tuesday, June 9, 2026

> ℹ Today: Tuesday June 9
>
> **Before class**
>
> - **Perusall Assignment**: *DC* §16.1–16.2, 16.4–16.5
>
> **Plan for today**
>
> - `read_csv()`
> - Data cleaning: NA values, column names, column types
> - Other file formats and reading from URLs
> - Writing data
> - Dates with lubridate
> - In-class activity
>
> **Helpful links**
>
> - Syllabus · AI policy · Schedule · *R4DS* Ch. 7 · *DC* Ch. 16 · readr reference

Getting data into R

Pre-loaded data

Every dataset we've used this semester was already in R: `penguins`, `flights`, `diamonds`. That won't be true in real work.

Real data lives in files, including CSV spreadsheets, Excel workbooks, tab-delimited exports from survey software, files someone emails you.

Today's goal: get a file off disk (or the web) and into a tidy data frame that we can easily work with.

> **Note**
>
> `readr` is part of the tidyverse, so `library(tidyverse)` is all you need for CSV and most text files.

read_csv()

The most common file type you'll encounter is **CSV** (comma-separated values), which can be read in using`read_csv()`:

```
students <- read_csv("data/students.csv")
```

We can also read in CSV files directly from a URL, without having to download anything:

```
students <- read_csv("https://pos.it/r4ds-students-csv")
```

```
Rows: 6 Columns: 5
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (4): Full Name, favourite.food, mealPlan, AGE
dbl (1): Student ID

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

After reading in the CSV, readr prints a **column specification message** that tells you how many rows and columns it found, and what type it guessed for each column.

Column specification message

```
Rows: 6 Columns: 5
  Column specification
Delimiter: ","
chr (4): Full Name, favourite.food, mealPlan, AGE
dbl (1): Student ID
```

- `chr` — character string
- `dbl` — double (numeric)
- `lgl` — logical (TRUE/FALSE)
- `dttm` / `date` — date-time / date

readr samples up to 1,000 rows and works through a set of rules to guess each column's type. It's usually right, but not always. The rest of today is largely about what to do when it's not.

What we actually got

```r
students
```

```
# A tibble: 6 x 5
  `Student ID` `Full Name`      favourite.food     mealPlan            AGE
         <dbl> <chr>            <chr>              <chr>               <chr>
1            1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
2            2 Barclay Lynn     French fries       Lunch only          5
3            3 Jayendra Lyne    N/A                Breakfast and lunch 7
4            4 Leon Rossini     Anchovies          Lunch only          <NA>
5            5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
6            6 Güvenç Attila    Ice cream          Lunch only          6
```

A few things to fix:

- `favourite.food` has `"N/A"` as a string, not a real `NA`
- `Student ID` and `Full Name` have spaces, which are awkward to type
- `AGE` is a character column even though it's mostly numbers (`"five"` snuck in)
- `mealPlan` is categorical and should be a factor

Problem 1: NA values

By default, readr only treats empty cells as `NA`. Strings like `"N/A"`, `"."`, `"missing"`, or `"-"` are left as-is.

Use the `na` argument to tell readr what else counts as missing:

```r
students <- read_csv(
  "https://pos.it/r4ds-students-csv",
  na = c("N/A", "")
)
```

```
Rows: 6 Columns: 5
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (4): Full Name, favourite.food, mealPlan, AGE
dbl (1): Student ID

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
students
```

```
# A tibble: 6 x 5
  `Student ID` `Full Name`      favourite.food     mealPlan            AGE
         <dbl> <chr>            <chr>              <chr>               <chr>
1            1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
2            2 Barclay Lynn     French fries       Lunch only          5
3            3 Jayendra Lyne    <NA>               Breakfast and lunch 7
4            4 Leon Rossini     Anchovies          Lunch only          <NA>
5            5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
6            6 Güvenç Attila    Ice cream          Lunch only          6
```

`favourite.food` row 3 is now `NA` instead of the string `"N/A"`.

Problem 2: column names

Columns with spaces are **non-syntactic**, and you can't type them without backticks:

```
students$Student ID   # Error
students$`Student ID` # Works, but annoying
```

Two fixes. Rename manually:

```
students |> rename(student_id = `Student ID`, full_name = `Full Name`)
```

Or use `janitor::clean_names()` to convert **everything** to snake_case at once:

```
students |> janitor::clean_names()
```

```
# A tibble: 6 x 5
  student_id full_name         favourite_food     meal_plan           age
       <dbl> <chr>             <chr>              <chr>               <chr>
1          1 Sunil Huffmann    Strawberry yoghurt Lunch only          4
2          2 Barclay Lynn      French fries       Lunch only          5
3          3 Jayendra Lyne     <NA>               Breakfast and lunch 7
4          4 Leon Rossini      Anchovies          Lunch only          <NA>
5          5 Chidiegwu Dunkel  Pizza              Breakfast and lunch five
6          6 Güvenç Attila     Ice cream          Lunch only          6
```

`janitor` isn't part of the tidyverse; run `install.packages("janitor")` once.

Problem 3: wrong column types

`mealPlan` is categorical, but we want to make it a factor. `age` is numeric, so we need adjust the (incorrect) character string `"five"` to be numeric first, then parse:

```
students <- students |>
  janitor::clean_names() |>
  mutate(
    meal_plan = factor(meal_plan),
    age = parse_number(if_else(age == "five", "5", age)) # vectorized ifelse()
  )
students
```

```
# A tibble: 6 x 5
  student_id full_name         favourite_food     meal_plan             age
       <dbl> <chr>             <chr>              <fct>               <dbl>
1          1 Sunil Huffmann    Strawberry yoghurt Lunch only              4
2          2 Barclay Lynn      French fries       Lunch only              5
3          3 Jayendra Lyne     <NA>               Breakfast and lunch     7
4          4 Leon Rossini      Anchovies          Lunch only             NA
5          5 Chidiegwu Dunkel  Pizza              Breakfast and lunch     5
6          6 Güvenç Attila     Ice cream          Lunch only              6
```

`if_else(test, yes, no)`: where `age == "five"`, substitute `"5"`; otherwise leave it alone. Then `parse_number()` converts the string to a number.

Tip

`parse_number()` is also great for values like `"$1,234"` or `"42%"` since it strips non-numeric characters automatically.

Column type detection

How readr guesses

For each column, readr samples up to 1,000 values and asks, in order:

1. Only `TRUE`, `FALSE`, `T`, `F`? → **logical**
2. Only numbers? → **double**
3. Matches ISO 8601 (`2026-06-09`)? → **date**
4. Otherwise → **character**

```r
read_csv("
  logical,numeric,date,string
  TRUE,1,2021-01-15,abc
  false,4.5,2021-02-15,def
  T,Inf,2021-02-16,ghi
")
```

```
Warning: The `file` argument of `read_csv()` should use `I()` for literal data as of
readr 2.2.0.

  # Bad (for example):
  read_csv("x,y\n1,2")

  # Good:
  read_csv(I("x,y\n1,2"))
```

```
Rows: 3 Columns: 4
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): string
dbl  (1): numeric
lgl  (1): logical
date (1): date
```

```
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# A tibble: 3 x 4
  logical numeric date       string
  <lgl>     <dbl> <date>     <chr>
1 TRUE        1   2021-01-15 abc
2 FALSE       4.5 2021-02-15 def
3 TRUE      Inf   2021-02-16 ghi
```

This works well with clean data, but fails when unexpected values like `"N/A"` or `"five"` are present.

Overriding types: col_types

We can tell readr what type to expect with the `col_types` argument:

```r
read_csv(
  file,
  col_types = list(
    student_id = col_integer(),
    meal_plan  = col_factor(),
    age        = col_double()
  )
)
```

Available column types:

Function	Type
`col_double()` / `col_integer()`	numbers
`col_character()`	always a string (good for IDs, phone numbers)
`col_logical()`	TRUE/FALSE
`col_factor()`	categorical
`col_date()` / `col_datetime()`	dates
`col_skip()`	drop this column entirely

When guessing goes wrong: problems()

If you force a type and readr finds values that don't fit, it warns you and stores the details:

```r
simple_csv <- "x\n10\n.\n20\n30"

df <- read_csv(simple_csv, col_types = list(x = col_double()))
```

```
Warning: One or more parsing issues, call `problems()` on your data frame for details,
e.g.:
  dat <- vroom(...)
  problems(dat)
```

```r
problems(df)
```

```
# A tibble: 1 x 5
    row   col expected actual file
  <int> <int> <chr>    <chr>  <chr>
1     3     1 a double .      /private/var/folders/_4/htlmsm_n50vcj9k6v2766k_40~
```

Row 3 has "." where readr expected a number. Now that we have identified the culprit, we can fix it:

```r
read_csv(simple_csv, na = ".")
```

```
Rows: 4 Columns: 1
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (1): x

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# A tibble: 4 x 1
      x
  <dbl>
1    10
2    NA
3    20
4    30
```

Tip

The workflow: read with forced types → `problems()` to locate oddities → decide whether they're true missing values or something else.

Other file formats

Other read_ functions

Once you know `read_csv()`, the others follow the same pattern, but have a different delimiter:

Function	Use when
`read_csv()`	comma-separated (`.csv`)
`read_csv2()`	semicolon-separated (common in Europe where `,` is the decimal)
`read_tsv()`	tab-delimited (`.tsv`, `.txt`)
`read_delim()`	any delimiter; guesses if you don't specify
`read_fwf()`	fixed-width fields

All of them accept a **file path or a URL** as the first argument.

```
# From disk
read_tsv("data/survey_export.txt")

# From the web - no download needed
read_csv("https://some-data-portal.gov/dataset.csv")
```

Reading from a URL

Any `read_*` function accepts a URL. This is useful for:

- Data portals that publish CSVs with a stable link
- Course data files hosted on GitHub or a course server
- Reproducibility: your script works on any machine without manual downloads

```
houses <- read_csv(
  "https://mdbeckman.github.io/dcSupplement/data/houses-for-sale.csv"
)
```

> **Note**
>
> If the URL is long, assign it to a variable first:
>
> ```r
> url <- "https://..."
> df  <- read_csv(url)
> ```

Writing data

write_csv()

To save a data frame to CSV:

```r
write_csv(students, "data/students-clean.csv")
```

The catch: column types are lost. When you read the file back, readr guesses types fresh and `meal_plan` goes back to character, not factor.

```r
read_csv("data/students-clean.csv")   # meal_plan is <chr> again
```

CSV is a plain-text format, so it stores values, not type metadata.

Preserving types: write_rds()

For saving intermediate results within R, use **RDS**, which is R's native binary format. In this format, types are preserved exactly.

```r
write_rds(students, "data/students-clean.rds")
read_rds("data/students-clean.rds")   # meal_plan is <fct>, age is <dbl>
```

Format	Readable outside R	Preserves types	File size
CSV	Yes	No	Medium
RDS	No	Yes	Small

Use CSV to share data with others or for archiving. Use RDS when you're saving an intermediate result for your own pipeline and care about types.

Data entry

tribble() and tibble()

Sometimes you need to key in a small table by hand. You have two options:

`tibble()`: column by column:

```
tibble(
  x = c(1, 2, 5),
  y = c("h", "m", "g"),
  z = c(0.08, 0.83, 0.60)
)
```

```
# A tibble: 3 x 3
      x y         z
  <dbl> <chr> <dbl>
1     1 h      0.08
2     2 m      0.83
3     5 g      0.6
```

`tribble()`: row by row (easier to read for small tables):

```
tribble(
  ~x, ~y,   ~z,
   1, "h", 0.08,
   2, "m", 0.83,
   5, "g", 0.60
)
```

```
# A tibble: 3 x 3
      x y         z
  <dbl> <chr> <dbl>
1     1 h      0.08
2     2 m      0.83
3     5 g      0.6
```

Column headers start with `~`. This is what we used to define `students` and `departments` for the join activity yesterday.

Cleaning data

Strings to numbers

Real data often stores numbers as strings, such as formatted currency, percentages, European decimals.

`as.numeric()`: works on clean numeric strings:

```r
as.numeric(c("42", "3.14", "NA"))
```

```
Warning: NAs introduced by coercion
```

```
[1] 42.00  3.14    NA
```

`parse_number()`: ignores non-numeric characters:

```r
parse_number(c("$1,234.56", "17%", "9.8 m/s"))
```

```
[1] 1234.56   17.00    9.80
```

`parse_number()` is more forgiving and usually the right choice for messy real-world values.

> Warning
>
> `as.numeric("nine")` returns `NA` with a warning. If you have words, fix them first with `if_else()` or `case_when()` before parsing.

Recoding integer codes

Survey exports and government data often store categories as integers. They will typically provide a codebook that tells you what the numbers mean.

Strategy: build a lookup table, then `left_join()`:

```r
fuel_codes <- tribble(
  ~code, ~fuel_type,
      2, "gas",
      3, "electric",
      4, "oil"
)

houses_raw <- tibble(fuel = c(2, 3, 2, 4))

houses_raw |> left_join(fuel_codes, join_by(fuel == code))
```

```
# A tibble: 4 x 2
   fuel fuel_type
  <dbl> <chr>
1     2 gas
2     3 electric
3     2 gas
4     4 oil
```

Dates with lubridate

Dates stored as strings need to be converted before R can do date arithmetic or plot them in order.

`lubridate` (part of the tidyverse, but needs `library(lubridate)`) provides functions named after the order of the components:

```r
library(lubridate)

mdy("June 9, 2026")          # month-day-year
```

```
[1] "2026-06-09"
```

```r
ymd("2026-06-09")            # ISO 8601
```

```
[1] "2026-06-09"
```

```r
dmy("9 June 2026")           # day-month-year
```

```
[1] "2026-06-09"
```

```r
mdy_hm("06/09/2026 09:35")   # with time
```

```
[1] "2026-06-09 09:35:00 UTC"
```

Once a date is a proper `<date>` object, arithmetic works as you would expect:

```r
ymd("2026-06-26") - ymd("2026-06-09")  # days until end of course
```

```
Time difference of 17 days
```

> **Note**
>
> Always check what order the components are in the original string. `mdy()` vs `dmy()` on the same string give different (wrong) dates.

Factors vs. character strings

R has a special type for categorical data: **factors**. They store categories efficiently and let you control the order of levels.

- `readr::read_csv()` reads text as **character** by default.
- The older `read.csv()` (base R) converts text to **factors** by default — this causes problems.

When to convert to factor:

```r
df |> mutate(region = factor(region))                         # unordered
df |> mutate(size = factor(size, levels = c("S","M","L")))  # ordered
```

Watch out: if you accidentally have a factor when you expect a string, `as.numeric()` gives the level number, not the value. Use `as.character()` first.

Activity

In-class activity

Work through `day16-activity.R` in RStudio.

- **Part 1**: read in and clean the students CSV step by step
- **Part 2**: houses-for-sale data: integer codes, missing values, writing out
- **Part 3**: Saratoga dates (challenge)

Activity file

Wrap-up & looking ahead

What we covered

- `read_csv()` and its key arguments: `na`, `col_types`, `col_names`
- `janitor::clean_names()`, `parse_number()`, `factor()`
- Other file types: `read_tsv()`, `read_delim()`, `read_csv2()`
- `write_csv()` vs `write_rds()`
- `tribble()` for hand-keyed tables
- Recoding integers via `left_join()`
- Dates with `lubridate`

Upcoming

- **Tomorrow (Wed Jun 10):** exam review
- **Thursday (Thu Jun 11):** in-class midterm exam
- **HW 4** due Mon Jun 15

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 7 · readr reference